

Computer Programming 1

Laboratory 07

(functions and parameter passing – part 2)

Exercise 1: Serie *pi*

- Write a program that calculates the result of the following series that approximates the value *pi* (π)
- The approximation value of this series N, is given by the user.

$$\sum_{i=1}^N \frac{1}{i^2} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \cdots + \frac{1}{N^2}$$

- Compute pi (π) by considering that $\sum_{i=1}^{\infty} \frac{1}{i^2} = \frac{\pi^2}{6}$

Constraint: use at least one additional function, with respect to the *main*, in your implementation. You can use the parameter passing by value

Constraint: do not use arrays

Exercise 2: Print uppercase (v2)

- Given a character in input, write a program that checks - with a function named *checkCharacter* - whether the character provided in input is a lowercase letter of the alphabet. If so, the program must print the uppercase character on the screen, using another function - named *convertCharacter* - for the conversion.

E.g., 'a' => 'A'

Implementation Constraints

- Do not use any global variables
- Use **passing by reference** for the function *convertCharacter*

Constraint: do not use arrays

Example 4: Liar's Dice (2)

- Write a program that:
 1. The program asks the user to guess how many dices turned out to be 1
 2. then it rolls 10 6-sided dices by using the function *rand()* of the library *cstdlib*
 3. If the number specified by the user corresponds to the number of 1 of the random dices, the user is the winner, and a message is printed.
 4. Otherwise, the number inserted by the user is wrong, a message is printed and the program execution ends

Implementation constraints:

- Use a procedure called `create_dice_roll` for the step 2.
- Do not use any global variables
- Use **passing by reference** for the procedure `create_dice_roll`

```
#include <cstdlib>
srand(time(NULL));
rand()
```

For additional details:

<http://www.cplusplus.com/reference/cstdlib/rand/>

Constraint: do not use arrays

Exercise 4: Swap

- Given in input two real numbers (`double`), in two variables, write a program that uses a procedure to exchange the values of the two variables using the passage of parameters by pointers.

Constraint: do not use arrays

Exercise 5: Division

Given in input two integers, write a program that makes the mathematical operation “division” by using a function. The function returns the quotient as return value, and the rest is returned by means of a reference parameter.

- In this program, you cannot use the symbols ‘/’ and ‘%’ for division (i.e., the division quotient and the rest needs to be computed in a different way)

Suggestion: you can use other mathematical operations

Constraint: use only `<iostream>` as library, not other libraries

Constraint: do not use arrays

Exercise 6: seconds/minutes/hours

- Given as input three integers in three variables (seconds, minutes, hours), write a program that, with a procedure, converts a possible excess of seconds into minutes and a possible excess of minutes into hours.
- Use parameter passing by reference and/or pointer.
- Use at least two additional functions, other than the main
- (if possible) Use the function “division” implemented in the previous exercise

sec = 121;		sec = 1;
min = 59;	=>	min = 1;
hour = 2;		hour = 3;

Constraint: do not use arrays

Exercise 7: Max (1)

- Given as input a set of 1 to 5 integers, write a program that returns the maximum value. Ask the user to insert the five integer values.

Constraint:

- Do not use any libraries apart `iostream` and `limits`
- Use only fundamental data types, do not use arrays or vectors or any complex data structure!
- In the implementation, use at least two additional functions, other than the `main()`.
- Use default arguments for the second, third, fourth, and fifth inputs.
- Check the `cin` by using `fail`, etc, for verifying the numbers inserted by the user

Constraint: do not use arrays

Exercise 7: Max (2)

- Given as input a set of 1 to 5 integers, write a program that returns the maximum value. Ask the user to insert the five integer values.

Constraint:

- Do not use any libraries apart `iostream` and `climits`
- Use only fundamental data types, do not use arrays or vectors or any complex data structure!
- In the implementation, use at least two additional functions, other than the `main()`.
- Use default arguments for the second, third, fourth, and fifth inputs.
- Check the `cin` by using `fail`, etc, for verifying the numbers inserted by the user

Variant: write a variant in which the program iteratively requires to the user when stop the execution

Constraint: do not use arrays

Exercise 8: Sort (1)

- Given as input three positive integers, in three variables (n1, n2, n3), write a program that, with a procedure, "re-orders" the numbers in ascending order by using the passage of parameters by reference.

n1 = 3;		n1 = 2;
n2 = 7;	=>	n2 = 3;
n3 = 2;		n3 = 7;

Implementation constraints:

- Ask the user to enter the integer numbers
- Do not use libraries (excepts iostream),
- Do not use arrays (or any other complex data structure).
- Define and use two additional functions (with respect to main) by using the following declarations:
void sort(int & n1, int & n2, int & n3);
void swap(int & n1, int & n2);

Constraint: do not use arrays

Exercise 9: RisiKo! 3vs3

Write a program that simulates a 3-on-3 attack on RisiKo!.

1. Roll 3 dices to 6 faces for the attacker and 3 dices for the defender.
2. Compare the highest dice of the attacker against the highest of the defender, the middle of the attacker against the middle of the defender and the lowest of the attacker against the lowest of the defender.
3. Finally, declare the number of clashes won by the attacker and the defender.

Constraints: it is requested to develop (at least) a version of the program that:

- It uses a function for roll the dice – by using random number –
- It uses three procedures respectively for sorting, swapping and comparing the rolled dices
- At least two out of the three procedures use the parameter passing by reference



Additional implementation constraints:

- Do not use libraries (excepts iostream and **cstdlib**)
- Do not use arrays (or any other complex data structure)

Constraint: do not use arrays

Exercise 10:Swap v2

- Given in input two integers (short), write a program that, with a procedure, exchanges the 8 bits less significant in the two numbers.
- It is requested to develop (at least) a version of the program that uses the passage of parameters by pointers for the procedure.

1855 (0000011110011111) 1816 (0000011100011000)
1048 (0000010000011000) $\Rightarrow$ 1087 (0000010000111111)

Implementation constraints:

- Do not use libraries (excepts iostream)
- Do not use arrays (or any other complex data structure)

Constraint: do not use arrays